SENIOR PROJECT AW5-2026

Multi-Tenant RAG-based Conversational AI for Software Development Lifecycle

Deliverable Set 1

Project Concept

Submitted to

School of Information, Computer and Communication Technology
Sirindhorn International Institute of Technology
Thammasat University

August 2026

by

Thumathorn Rungratana 6622770293

Nathaporn Prucksayanont 6622771846

Nattapat Lertsiritam 6622772703

Peerawit Amnartpanich 6622780169

Advisor: Dr. Apichon Witayangkurn

Contents

Contents i

Project Concept 1

1.1 Summary 1

1.2 Motivation 1

1.3 Users and Benefits 2

1.4 Typical Usage 2

1.5 Main Challenges 3

Chapter 1

Project Concept

1.1 Summary

This project aims to design and develop a multi-tenant RAG-based conversational AI system to support information access throughout the Software Development Lifecycle (SDLC). The system allows users to upload and manage software project information, such as requirements, system features, database information, technical documents, and other project-related files created during different stages of software development.

Using Retrieval-Augmented Generation (RAG), the system retrieves relevant information from the selected project's knowledge base and provides it as context to a large language model to generate answers to users' questions. The system is designed to support project members in accessing information related to activities such as requirement analysis, development, testing, and maintenance.

Data from different tenants and projects are kept separate to prevent information from being mixed or exposed across projects. The system is also designed to answer based on the retrieved project information and inform users when sufficient information is unavailable.

1.2 Motivation

Software development projects generate a large amount of information throughout different stages of the SDLC, including requirements, system design information, implementation details, testing documents, database information, and maintenance-related records. As a project grows, this information may be stored across different files and locations, making it difficult and time-consuming for team members to find the information they need.

This project therefore proposes a conversational AI system that allows users to access project information through natural-language questions. Retrieval-Augmented Generation (RAG) is used so that the system retrieves relevant information from the selected project before generating an answer. This allows the language model to respond based on actual project information rather than relying only on its general knowledge.

The system also aims to reduce unsupported answers by informing users when sufficient information cannot be found in the project data. By providing easier access to project knowledge, the proposed system can support information retrieval across multiple stages of the software development process.

1.3 Users and Benefits

The primary users of the system are authorized members of software development projects who need to access information related to different stages of the SDLC. Users may require information about requirements, system features, database structures, implementation details, testing information, or maintenance-related documents.

The system provides a convenient way to access project knowledge without manually searching through multiple files and documents. It can also provide project-specific answers by retrieving information from the selected project's knowledge base.

In addition, the multi-tenant design keeps information from different tenants and projects separated, helping prevent project data from being mixed or accessed by unauthorized users. Overall, the system aims to make project information easier to access throughout the software development process.

1.4 Typical Usage

A typical usage begins when an authorized user accesses the system and selects a software project. Project-related documents from different stages of the SDLC, such as requirements, technical documents, database information, testing documents, or maintenance records, can be uploaded to the system and processed as part of the project's knowledge base.

When the user asks a question, the system searches the selected project's knowledge base for relevant information. The retrieved information is then combined with the user's question and provided to the large language model. The model generates an answer based on the retrieved project information.

For example, a user may ask about a system requirement, an existing feature, database information, testing details, or maintenance information. If sufficient information cannot be found, the system informs the user that the requested information is unavailable instead of generating an unsupported answer.

The overall process can be summarized as:

Upload Project Data → Process and Store Data → Select Project → Ask Question → Retrieve Relevant Information → Generate Answer → Display Response

1.5 Main Challenges

One of the main challenges of this project is handling different types of information produced throughout the SDLC. Software project data may include text documents, PDF files, requirements, diagrams, structured data, database information, testing documents, and maintenance records. Different types of data may require different processing methods, so the supported input types must be clearly defined within the scope of the project.

Another challenge is retrieval accuracy. The system must retrieve information that is relevant to the user's question from the correct project knowledge base. Poor retrieval results may cause the language model to generate incomplete or incorrect answers.

Data separation is also important in a multi-tenant environment. Information belonging to one tenant or project must not be mixed with or retrieved from another tenant or project. The system therefore needs to correctly associate users, projects, and stored information.

Finally, the system must reduce hallucination in generated answers. The language model should answer based on the retrieved project information and indicate when the available information is insufficient rather than generating an unsupported response.